Panduan Membuat Fitur — iOS

Manote · Swift, SwiftUI, MVVM · contoh acuan: fitur login

---

1. Gambaran alur (MVVM)

View → **ViewModel** → **Repository** → **APIClient** → **Backend**

Lapisan	Tugasnya	Contoh di fitur login
View	Tampilan SwiftUI. Meneruskan aksi pengguna ke ViewModel. Tidak memanggil API.	`LoginView`
ViewModel	State layar (isi form, loading, pesan error) dan logikanya.	`LoginViewModel`
Repository	Sumber data fitur: memanggil API dan penyimpanan lokal.	`AuthRepository`
Model	Bentuk data, `Codable` agar cocok dengan JSON backend.	`User`, `LoginRequest`, `LoginResponse`
Core	Dipakai semua fitur.	`APIClient`, `TokenStore`, `KeychainStore`
Sesi	Status login untuk seluruh aplikasi.	`SessionStore`

---

2. Peta file fitur login

```
ios/Manote/
├── App/
│   ├── ManoteApp.swift
│   ├── RootView.swift              memilih LoginView atau MainTabView
│   └── SessionStore.swift          loading / loggedOut / loggedIn(user)
├── Core/
│   ├── Networking/                 APIClient, APIError (+ APICode), APIConfig
│   └── Storage/                    KeychainStore, TokenStore
└── Features/
    ├── Auth/
    │   ├── Models/                 User, LoginModels
    │   ├── Repositories/AuthRepository.swift
    │   ├── ViewModels/LoginViewModel.swift
    │   └── Views/LoginView.swift
    └── Dashboard/                  DashboardView, DashboardViewModel
```

> File Swift baru yang disimpan di dalam folder `ios/Manote/` otomatis masuk ke proyek Xcode. Tidak perlu menambahkannya secara manual ke target.

---

3. Langkah membuat fitur baru

Contoh yang dipakai: fitur **edit profil**, dengan endpoint backend `PUT /api/profile`.

Langkah 1 — Pastikan kontrak API

Lihat panduan backend. `APIClient` sudah punya `get`, `post`, `put`, dan `delete`, jadi endpoint PUT ini bisa langsung dipanggil.

Langkah 2 — Model

```
// Features/Profile/Models/UpdateProfileRequest.swift
struct UpdateProfileRequest: Encodable {
    let fullName: String
}
```

Nama properti harus sama persis dengan nama field JSON dari backend. Response memakai ulang `User`.

Langkah 3 — Repository

```
// Features/Profile/Repositories/ProfileRepository.swift
struct ProfileRepository {
    var api = APIClient()

    func updateProfile(fullName: String) async throws -> User {
        try await api.put("/api/profile", body: UpdateProfileRequest(fullName: fullName))
    }
}
```

Langkah 4 — ViewModel

```
// Features/Profile/ViewModels/ProfileViewModel.swift
import Observation

@MainActor
@Observable
final class ProfileViewModel {
    var fullName: String
    private(set) var isLoading = false
    private(set) var errorMessage: String?

    private let repository = ProfileRepository()

    init(user: User) {
        fullName = user.fullName
    }

    func save() async -> User? {
        isLoading = true
        errorMessage = nil
        defer { isLoading = false }

        do {
            return try await repository.updateProfile(fullName: fullName)
        } catch {
            errorMessage = error.localizedDescription
            return nil
        }
    }
}
```

Pola ini sama dengan `LoginViewModel`: tiga state (`isLoading`, `errorMessage`, isi form) dan satu fungsi `async` yang mengembalikan hasil.

Langkah 5 — View

```
// Features/Profile/Views/ProfileView.swift
import SwiftUI

struct ProfileView: View {
    @Environment(SessionStore.self) private var session
    @State private var viewModel: ProfileViewModel

    init(user: User) {
        _viewModel = State(initialValue: ProfileViewModel(user: user))
    }

    var body: some View {
        @Bindable var viewModel = viewModel

        Form {
            TextField("Nama lengkap", text: $viewModel.fullName)
            if let errorMessage = viewModel.errorMessage {
                Text(errorMessage).foregroundStyle(.red)
            }
            Button(viewModel.isLoading ? "Menyimpan..." : "Simpan") {
                Task {
                    if let user = await viewModel.save() {
                        session.didLogIn(user)   // perbarui user di seluruh aplikasi
                    }
                }
            }
            .disabled(viewModel.isLoading)
        }
        .navigationTitle("Edit Profil")
    }
}
```

Langkah 6 — Sambungkan ke navigasi

Layar baru harus bisa dibuka dari suatu tempat. Dua cara yang umum:

- **Dari layar lain:** tambahkan `NavigationLink("Edit Profil") { ProfileView(user: user) }`, misalnya di `DashboardView`.
- **Sebagai tab baru:** tambahkan ke `MainTabView` di `RootView.swift`.

Langkah 7 — Jalankan dan uji

1. Nyalakan backend (`./mvnw spring-boot:run` di folder `backend`).
2. Buka `ios/Manote.xcodeproj` di Xcode, pilih simulator iPhone, tekan Run (⌘R).
3. Coba kasus berhasil, input salah, dan backend mati (pesan error harus muncul).

---

4. Penanganan error

Backend selalu menjawab dengan bentuk `{ status, title, code, data }`. `APIClient` membuka bungkusnya: yang dikembalikan adalah isi `data`, dan jika `status` bernilai `false` ia melempar `APIError` yang membawa `code` dan pesan dari `title`.

Aturannya: setiap error memunculkan **toast** secara otomatis. Fitur baru tidak perlu menulis apa pun untuk itu. Pengecualiannya hanya jika layar menampilkan error-nya sendiri.

Kasus	Yang terjadi otomatis	Yang Anda tulis di fitur
Error dari aksi pengguna (hapus, simpan)	Toast berisi pesan dari `title`	Tidak ada
Layar menampilkan error sendiri (pesan di form, daftar dengan Coba lagi, indikator simpan)	Tidak ada toast	Bungkus panggilan dengan `handlingErrorsLocally { ... }`, lalu tampilkan `error.localizedDescription`
Lebih dari 30 detik tanpa jawaban	Permintaan dihentikan dan toast muncul, walaupun dibungkus `handlingErrorsLocally`	Tidak ada
Sesi berakhir (4012)	Token dibuang dan aplikasi kembali ke layar login dengan pesan Sesi berakhir	Tidak ada
Butuh tindakan khusus per kode	—	Bandingkan `error.code` dengan `APICode`

```
// Bawaan: gagal -> toast otomatis
try? await repository.delete(id: note.id)

// Layar menampilkan error sendiri -> tanpa toast
notes = try await handlingErrorsLocally { try await repository.list() }
```

> Sheet menutupi layar di bawahnya, termasuk toast. Tambahkan `.toastHost()` pada isi setiap sheet baru supaya toast tetap terlihat. Untuk menahan layar selama permintaan berjalan, pakai `.loadingOverlay(isLoading, text:)`.

```
do {
    try await repository.updateProfile(fullName: fullName)
} catch let error as APIError where error.code == APICode.notFound {
    // tindakan khusus untuk kode ini
} catch {
    errorMessage = error.localizedDescription
}
```

Hal yang perlu diingat

- Alamat backend ada di `APIConfig`: simulator memakai `localhost`, iPhone asli memakai `ManoteDeviceAPIBaseURL` di `Manote-Info.plist`.
- Daftar kode ada di `APICode` (`Core/Networking/APIError.swift`) dan harus sama dengan `ErrorCode` di backend.
- HTTP tanpa HTTPS hanya diizinkan untuk jaringan lokal (`Manote-Info.plist`). Server publik wajib HTTPS.

- Token disimpan di Keychain lewat `TokenStore`. Jangan simpan token di `UserDefaults`.
- ViewModel diberi `@MainActor` agar perubahan state selalu terjadi di thread UI.

---

5. Checklist fitur iOS

- ☐ Model `Codable` cocok dengan JSON backend.
- ☐ Repository dibuat; View tidak memanggil `APIClient` secara langsung.
- ☐ ViewModel `@MainActor @Observable` punya state loading dan error.
- ☐ View menampilkan keadaan memproses dan pesan error.
- ☐ Layar bisa dibuka dari navigasi atau tab.
- ☐ Build berhasil dan fitur dicoba di simulator.